

TD n°3 : Un peu plus de Prolog

Choisissez votre éditeur préféré pour créer un fichier avec une extension `.pl` dans lequel vous écrirez votre programme, par exemple `tp3.pl`. Attention à ne pas utiliser de majuscule pour la première lettre du nom du fichier. Placez-vous dans le répertoire où figure votre fichier puis lancez Prolog grâce à la commande `pl`. Chargez votre programme grâce à l'instruction `[tp3]`. À chaque modification du programme, n'oubliez pas d'enregistrer et de recharger le fichier dans Prolog.

Si Prolog n'est pas installé dans votre machine, utilisez GNU Prolog sur la machine virtuelle Linux qui se trouve à l'adresse URL `m-vdi-etu-ha.univ-amu.fr`. Pour compiler un fichier `nom_du_fichier.pl`, utilisez la commande `gplc nom_du_fichier.pl` sur le terminal (dans le répertoire où figure votre fichier). Ça va créer l'exécutable `nom_du_fichier` que vous pouvez lancer en tapant `./nom_du_fichier` sur le terminal (dans le répertoire où figure votre fichier).

Exercice 1. Reprenez l'ensemble de faits qui définissait l'arbre généalogique de la famille de l'exercice 1 du TD1.

<code>homme(albert).</code>	<code>femme(marie).</code>	<code>pere(louis,benoit).</code>	<code>mere(marie,sophie).</code>
<code>homme(jean).</code>	<code>femme(germaine).</code>	<code>pere(albert,jean).</code>	<code>mere(marie,bertrand).</code>
<code>homme(paul).</code>	<code>femme(sophie).</code>	<code>pere(jean,simone).</code>	<code>mere(germaine,jean).</code>
<code>homme(bertrand).</code>	<code>femme(christiane).</code>	<code>pere(paul,sophie).</code>	<code>mere(christiane,simone).</code>
<code>homme(louis).</code>	<code>femme(simone).</code>	<code>pere(jean,paul).</code>	<code>mere(christiane,paul).</code>
<code>homme(benoit).</code>		<code>pere(paul,bertrand).</code>	<code>mere(simone,benoit).</code>

Dans l'ensemble de faits ci-dessus, il y a une certaine redondance d'information, ce qui n'est pas optimal. En fait, comme on a les prédicats `homme` et `femme`, on peut utiliser tout simplement le prédicat `parent(X,Y)` pour dire que X est parent de Y , et déduire la définition des prédicats `pere(X,Y)` et `mere(X,Y)` à partir d'un ensemble de faits qui n'utilise que les prédicats `homme`, `femme`, `parent`.

1. Réécrivez l'ensemble de faits ci-dessus en utilisant que les prédicats `homme`, `femme` et `parent`, pour définir le même arbre généalogique de la famille.
2. Définissez les prédicats suivants :
 - (a) `pere(X,Y)` : X est le père de Y .
 - (b) `mere(X,Y)` : X est la mère de Y .
 - (c) `grand_parent(X,Y)` : X est un grand parent de Y .
 - (d) `avoir_un_enfant(X)` : X a au moins un enfant.
 - (e) `etre_un_enfant(X)` : X est l'enfant de quelqu'un.
 - (f) `oncle(X,Y)` : X est un oncle de Y .
 - (g) `tante(X,Y)` : X est une tante de Y .
 - (h) `cousin(X,Y)` : X est un cousin de Y .
 - (i) `cousine(X,Y)` : X est un cousin de Y .
 - (j) `ancetre(X,Y)` : X est un ancêtre de Y . *Suggestion* : Utilisez la récursion ! Le fait X est un ancêtre de Y signifie que ou bien X est un parent de Y , ou bien X est un parent d'ancêtre de Y .
 - (k) `marie(X,Y)` : X et Y sont mariés (on suppose que pour être mariés, il suffit d'avoir un enfant en commun).

Exercice 2. Reprenez l'exercice 4 du TD2. Pour chacun des points de cet exercice, faites les choses suivantes :

1. Réécrivez les ensembles de formules données en tant qu'ensemble de faits et règles en Prolog.
2. Vérifiez que Prolog va répondre **true** ou **yes** à la conséquence qu'on demande de déduire de l'ensemble de faits et règles.

Exercice 3. Considérez l'ensemble de faits suivant sur Prolog :

```
sign(X, negative) :- X < 0.  
sign(X, zero) :- X == 0.  
sign(X, positive) :- X > 0.
```

1. Observez la réponse donnée par Prolog si vous lui posez la question suivante :
`?- sign(-1,Y).`
Qu'est-ce qu'il se passe ? Pourquoi Prolog ne termine pas l'exécution après avoir répondu `Y = negative` ? Est-ce qu'on peut dire que l'ensemble de faits ci-dessus décrit un programme Prolog efficace ?
2. Utilisez le cut `!`, en l'ajoutant à l'ensemble de faits ci-dessus, pour rendre le programme Prolog plus efficace, sans changer son output. Notamment, on voudrait que l'exécution termine tout de suite dès que Prolog répond à chacune des questions suivantes :
`?- sign(-1,Y).`
`?- sign(0,Y).`
`?- sign(1,Y).`

Exercice 4. Supposons que Prolog connaît uniquement le fait `rond(sphere)`.

sans aucune règle. On peut poser à Prolog la question suivante :

```
?- rond(terre).
```

et il va nous répondre **false** ou **no** parce que Prolog ne connaît pas le fait `rond(terre)` et il ne peut pas le déduire étant donnés les faits (`rond(sphere)`) et règles (aucune) que Prolog connaît. Donc, Prolog va répondre **true** ou **yes** à la question suivante, parce que `\+` est le prédicat de négation en Prolog.

```
?- \+(rond(terre)).
```

(La morale de tout ça étant que tout ce que Prolog ne peut pas prouver à partir des faits et règles qu'on lui a donné, est faux pour Prolog.)

1. Écrivez la règle définissant le prédicat **nonrond** comme la négation du prédicat **rond**, sans utiliser `\+` mais en utilisant le cut `!`, les prédicats 0-aires **fail** et **true** et la variable anonyme `_` ("any").
2. Définissez un prédicat **neg(X)** pour la négation : si **X** est vrai, alors **neg(X)** renvoie **false** ou **no** ; et si **X** est faux, alors **neg(X)** renvoie **true** ou **yes**.
3. Testez que **neg(rond(X))** a effectivement le même comportement que **nonrond(X)**, pour plusieurs valeurs de la variable **X** (par exemple, **sphere** et **terre**).